

Lab A11 - Python Dictionaries

Lab Instructions

Why Use Dictionaries?

In programming, we often need to store and organize **related data**. A **dictionary** lets us pair each piece of information with a **meaningful key**, making the data:

- Easier to read
- Easier to search
- Easier to update

For example:

```
student = {  
    "name": "Alice",  
    "age": 19,  
    "major": "Mathematics"  
}
```

You can access "Alice" directly with `student["name"]` instead of remembering an index like `student[0]`. This mirrors how you'd label fields in a form or columns in a spreadsheet.

Lab Objectives

For this lab, your goal is to implement all of the functions inside of `student_database.py`. In order for a TA to check your work, make sure to also download the `database_runner.py` file. This provides you with a simple command line program to run your functions and see if they're working.

What Do I Turn In?

Turn in your modified `student_database.py` file to the appropriate place on eClass after checking your code off with a TA or the instructor.

Understanding Dictionaries – A Quick Primer

Example 1: Basic Dictionary

```
# A simple dictionary representing a student
student = {
    "name": "Alice",
    "age": 19,
    "major": "Mathematics"
}

# Accessing values using keys
print(student["name"]) # Alice

# Adding or updating values
student["gpa"] = 3.8
student["age"] = 20
```

Example 2: Dictionary with Lists

```
# Each subject maps to a list of grades
grades = {
    "Algebra": [95, 88, 92],
    "Calculus": [89, 94]
}

# Accessing nested values
print(grades["Algebra"][0]) # 95
```

Example 3: Nested Dictionary

```
# Database of students, each with a unique ID
students = {
    "s101": {"name": "Alice", "major": "Math"},
    "s102": {"name": "Bob", "major": "Physics"}
}

# Access a specific student's name
print(students["s102"]["name"]) # Bob
```